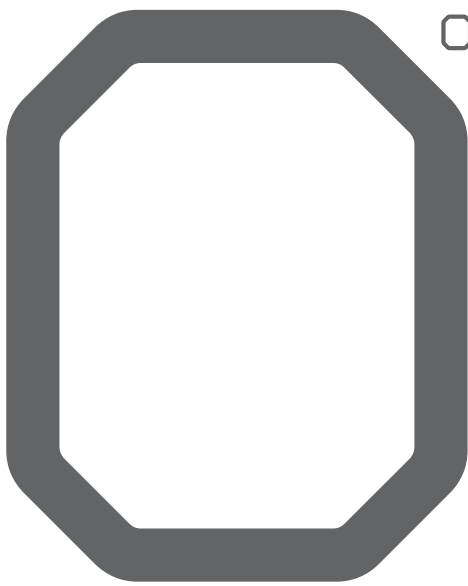




Opinion

AI Didn't Make Programming Easier. It Just Made It Differently Difficult

The future of software development will belong to those who can think clearly at scale, maintain durable mental models amid rapid change, and integrate machine-generated output into human-directed intent.

FOR DECADES, EMPIRICAL research has shown that programming is a demanding cognitive activity: Developers rely on working memory, long-term recall, and complex mental models to manipulate interacting abstractions, such as control flow, data structures, and the structural design of software. This classical model frequently positioned memory and recall as both the central enablers of, and bottlenecks in, software development.

Today's AI-powered coding assistants are changing that. These tools function as external memory systems, offloading syntax recall, boilerplate generation, and API usage from human to machine memory. As memory demands lessen, reasoning, architectural comprehension, judgment, and code-structure awareness are becoming comparatively more important.

Thus, "knowing how to program" is being fundamentally redefined, but

not in ways that make programming easier or that devalue the programmer. This article describes four major shifts in how programming work is changing:

1. The field is opening to new practitioners.
2. The work is becoming differently difficult.
3. Education is transforming.
4. The programmer's role is evolving from knowledge vessel to orchestrating agent.

Together, these shifts suggest that AI is not eroding the cognitive substance of programming but relocating it—making different skills matter more and creating new forms of difficulty even as old barriers fall.

Programming as High-Memory Work

For most of the history of software development, researchers found that programming relied on a combination

of reasoning ability, working memory capacity, and long-term memory recall. Brooks described programming as a cognitive process that depends on maintaining multiple interacting abstractions.⁴ Pennington then demonstrated that programmers rely on complex mental models of control flow and data flow to design, understand, and modify code.⁶ Studies have shown that these mental models exist in the mind as abstractions, are maintained at high cognitive cost, and are relatively fragile. They break down easily under conditions of context switching and are costly to rebuild in cognitive terms if they are not documented.

Today, however, AI coding assistants are reshaping this traditionally onerous cognitive landscape. Developers now have access to external memory systems capable of generating code, retrieving syntax, reconstructing context, and remembering and regenerating variants of previous mental con-

structs. This does not eliminate the need to think. But it does shift which cognitive skills are becoming most relevant for programmers.

Working memory and long-term recall. Working memory is the limited-capacity system that supports temporary storage and manipulation of discrete blocks of information, as formalized by Baddeley and Hitch.² Long-term memory, conversely, serves as the repository of consolidated knowledge, including declarative information, such as syntax and architectural concepts, and procedural knowledge, such as coding idioms and problem-solving schemas.

Programming has traditionally relied heavily on both. Soloway, Bonar, and Ehrlich found that programmers rely on internal cognitive preferences,⁹ presumably based on experience and retrieved from long-term memory, to structure their approach to iterations. For example, when a loop construct matched the programmer's natural plan, correctness increased dramatically. This suggests that programming has historically been guided not just by syntax knowledge but by the formation and execution of internal cognitive preferences retrieved from memory and expressed as mental models through design choices. Siegmund et al. used fMRI to show that code comprehension activates networks associated with working memory, attention, and language processing,⁸ providing physiological evidence that understanding complex programming tasks is a resource-intensive cognitive task at the neurological level.

Theories of AI as an external memory resource. AI coding assistants alter this cognitive landscape by acting as external memory and cognition. This aligns with Hutchins' theory of distributed cognition, which argued that cognitive systems often extend beyond the individual to include the external environment,⁵ which can extend and enhance individual cognition.

Alternate theories complement distributed cognition. Cognitive load theory (CLT) holds that AI tools can reduce extraneous load, which is the memory overhead of recalling syntax and boilerplate, thereby freeing working memory for intrinsic, high-level



reasoning. The Extended Mind Hypothesis (EMH) goes further by focusing on the individual: If an AI assistant becomes reliably available, habitually used, and trusted, it can function as an integrated component of the programmer's cognitive architecture rather than as an external tool. Under this view, the AI becomes part of the thinking process itself, shaping reasoning, decision making, and the effects of cognitive effort.

Barke, James, and Polikarpo-

The AI is not merely a faster search engine; it is at least partly an integrated extension of human cognitive architecture.

va³ showed that developers commonly use Copilot to offload low-level work, such as typing boilerplate, recalling API details, and looking up unfamiliar syntax, while shifting effort toward validating and integrating the generated code. These findings collectively establish that the AI is not merely a faster search engine; it is at least partly an integrated extension of human cognitive architecture. These findings collectively support the view that AI is not merely a faster search engine; it is at least partly an integrated extension of human cognitive architecture.

What AI removes and what it does not. Thus, AI fundamentally changes the costs associated with imperfect memory, effectively reducing the penalty for imperfect recall. A developer can successfully request a common API usage pattern without precise internal recall or retrieve complex syntax without relying on working-memory-intensive reconstruction. This capacity directly reduces the dependency on the rapid retrieval of specific, low-level knowledge from long-term memory,



mitigating the cognitive bottleneck previously identified.

However, while AI-assisted programming tools can accelerate routine development tasks, they do not eliminate the need for human oversight, particularly when work requires conceptual reasoning rather than surface-level code manipulation. Every programmer knows AI can produce code that is syntactically correct yet semantically and subjectively flawed, meaning developers must still understand program structure well enough to detect errors, evaluate coding suggestions critically, and ask the necessary “why” and “why not” questions about causal behavior.

Shihab et al. found that students using GitHub Copilot completed brown-field tasks substantially faster and with more solution progress, but in exit interviews many reported concerns about not fully understanding how or why Copilot’s suggestions worked, and the authors call for pedagogical approaches that leverage Copilot’s benefits while fostering comprehen-

The emerging reality is not that AI replaces the programmer, but that it becomes a second cognitive engine running in parallel.

sion.⁷ Alanazi et al., in a meta-analysis of controlled studies of tools such as ChatGPT and Copilot in programming education, reported that while AI assistance improves task performance and efficiency, it offers only small and statistically unstable gains in learning success and ease of understanding.¹

Taken together, these findings support the view that architectural reasoning, impact analysis, and long-term system maintenance cannot be offloaded to AI. They depend on deep, structural understanding of the codebase that remains the programmer’s responsibility. So, while AI may extend cognition and make certain programming tasks more efficient, thereby improving the productivity of trained developers, critical tasks such as debugging, refactoring, and systems analysis still require expertise and comprehension, and rely heavily on internal mental models that allow programmers to simulate execution and infer complex cause-and-effect paths within a codebase.

This marks a significant cognitive reorganization: Memory becomes a shared resource spanning human and machine; programming becomes less about what the developer can hold and manipulate in their mind and more about how clearly they can think at multiple scales when creating and ordering a complex logical system. Studies confirm that stable mental models of a codebase are essential for navigation and reasoning. Therefore, if developers outsource too much thinking to AI, those internal models can weaken. AI thus shifts cognitive load rather than removing it and speeds up writing code but increases the time spent checking and validating it.

In other words, the hard part moves from recall (“How do I write this?”) to judgment (“Does this actually make sense?”). This shift from recall-based to judgment-based programming represents the fundamental cognitive transformation at the heart of AI-assisted development. Where traditional programming demanded that developers maintain vast internal libraries of syntax, patterns, and idioms, AI-enabled programming demands instead they maintain robust evaluative frameworks for assessing correctness, coherence, and appropriateness.

While AI-assisted programming tools can accelerate routine development tasks, they do not eliminate the need for human oversight.

The cognitive burden has not disappeared—it has relocated from retrieval to reasoning.

Programming as Hybrid Cognitive Systems Work

The emerging reality is not that AI replaces the programmer, but that it becomes a second cognitive engine running in parallel. The machine surfaces patterns, stitches together APIs, retrieves forgotten syntax, and drafts first-pass solutions at a speed that reduces low-level recall bottlenecks but can also increase the burden of evaluation. The human, in turn, becomes less a conductor of keystrokes and more a shaper of intent: checking, interpreting, integrating, synthesizing, validating, reorganizing, discarding, editing, and ultimately giving structure and purpose to what the system supplies. It resembles what has already happened in medicine: Diagnostic tools lowered the burden of memorizing obscure clinical details but raised the stakes for interpretation, judgment, and error detection.



Expertise became less about recall and more about reasoning.

Understanding this hybrid arrangement helps explain why AI makes programming differently difficult rather than simply easier. The difficulty has not been eliminated; it has been redistributed across a new cognitive architecture that spans human and machine. This redistribution creates new challenges even as it removes old ones.

As AI continues to function primarily as an externalized memory and a code-transformation layer, four major shifts are already visible:

1. First, the field will open. Many who once would have bounced off the sheer cognitive overhead of memorizing libraries, syntax variations, or error-handling idioms will now find a workable entry path. The bottleneck of recall shrinks, making programming more accessible to people who might previously have struggled with syntax, library details, or unfamiliar idioms, while placing greater emphasis on problem decomposition, systems reasoning, and the ability to evaluate generated code. However, this accessibility comes with a paradox: While the barrier to producing code lowers, the barrier to producing good code may actually rise, as judgment becomes more critical and harder to develop than recall ever was.

2. Second, the work does not become simpler, only differently difficult. Conceptual clarity, decomposition, debugging, and architectural foresight still demand effort, and perhaps more of it. The challenge moves up a level. Where novice programmers once struggled primarily with syntax errors and API usage, they now struggle with evaluating whether AI-generated solutions are appropriate, maintainable, and aligned with broader system constraints. This represents a more sophisticated form of difficulty, one that requires deeper understanding of software-engineering principles rather than surface-level language features.

3. Third, education will shift. We will teach fewer people to memorize syntax and more to think in complex systems. The curriculum bends toward architecture, interface design, state management, failure modes,



constraint negotiation, test construction, security, and long-term maintainability. Code becomes only one representation of thought among many overlapping ones. Developers must still possess the skills to analyze, adapt, and modify what AI produces, perhaps even more so than before.

4. Fourth, and most importantly, the programmer remains essential. Not as a vessel of knowledge but as the orchestrating agent who understands the parts and how they fit together, maintains the integrity of the system, and who decides what matters. This shift has profound implications for how we understand programming expertise. The most capable developers of this new era will not be those who type the fastest or remember the most, but those who can hold deep mental models while offloading everything that interferes with that. They will combine strong systems reasoning with AI-augmented recall and will

The most capable developers of this new era will combine strong systems reasoning with AI-augmented recall and will treat the model almost like a cognitive prosthetic.

treat the model almost like a cognitive prosthetic: useful, fast, but incapable of finally determining subjective-semantic correctness or coherence.

Conclusion

Taken together, these four shifts suggest AI is not eroding the cognitive substance of programming but relocating it. As external memory becomes abundant and code generation cheap, the value of the programmer moves toward interpretation, structural reasoning, and judgment. The future of software development will belong to those who can think clearly at scale, maintain durable mental models amid rapid change, and integrate machine-generated output into human-directed intent.

AI does not, therefore, diminish the craft; it widens it, deepens it, and makes more of the work explicitly intellectual. The work becomes differently difficult because it demands more sophisticated forms of expertise: judgment over recall, architecture over syntax, orchestration over implementation. These are not easier skills to develop or demonstrate, they are simply different ones, and perhaps ultimately more demanding.

References

1. Alanazi, M., Soh, B., Samra, H., and Li, A. The influence of artificial intelligence tools on learning outcomes in computer programming: A systematic review and meta-analysis. *Computers* 14, 5 (2025), 185.
2. Baddeley, A.D. and Hitch, G. Working memory. In *The Psychology of Learning and Motivation* 8, Academic Press. G.H. Bower, ed. (1974), 47–89.
3. Barke, S., James, M.B., and Polikarpova, N. Grounded Copilot: How programmers interact with code-generating models. In *Proceedings of the ACM on Programming Languages* 7, Article 78 (2023), 85–111.
4. Brooks, R.E. Towards a theory of the cognitive processes in computer programming. *Intern. J. Man-Machine Studies* 9, 6 (1977), 737–751.
5. Hutchins, E. *Cognition in the Wild*. MIT Press (1995).
6. Pennington, N. Stimulus structures and mental representations in expert comprehension of computer programs. *Cognitive Psychology* 19, 3 (1987), 295–341.
7. Shihab, M.I.H. et al. The effects of GitHub Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield programming tasks. In *Proceedings of the ACM Conf. Inter. Computing Education Research* (2025), 407–420.
8. Siegmund, J. et al. Understanding source code with functional magnetic resonance imaging. In *Proceedings of the 36th Annual Intern. Conf. Software Engineering*, ACM (2014), 378–389.
9. Soloway, E., Bonar, J., and Ehrlich, K. Cognitive strategies and looping constructs: An empirical study. *Commun. ACM* 26, 11 (1983), 853–860.

Jeremy Osborn (jeremy.osborn@gmail.com) is a technology leader whose work spans software, governance, sustainability, and strategy.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2026 Copyright held by the owner/author(s).